

Informe Técnico de Sistema de Gestión de Rutas de Transporte Público

Pontificia Universidad Católica Madre y Maestra (PUCMM)

Asignatura: Algoritmos Clásicos y Estructuras de Datos

Integrantes del Proyecto:

Abraham Miller García

Yodariz Nazareth Mejías Román

Introducción

Este proyecto consiste en una aplicación orientada a objetos diseñada para optimizar la movilidad urbana. Utiliza una arquitectura de **Grafos Dirigidos** para modelar paradas y rutas, permitiendo el cálculo de trayectos óptimos basados en tiempo, costo y distancia.

Definición de la Estructura de Datos

Para cumplir con los requisitos de eficiencia en redes densas, se ha implementado el grafo utilizando **Listas de Adyacencia**. Sumado a esto para la estructura del proyecto se crearon cuatro directorios principales para manejar la lógica, la persistencia, el apartado visual y las pruebas de funcionamiento de la lógica de formas separadas. Cabe señalar que dentro del directorio LogicPack (directorío que contiene las entidades principales) se creó también una sección para las clases de los algoritmos y la interfaz AlgoZed, la cuál es implementada por las diferentes clases que representan los algoritmos.

Capa de Lógica (LogicPack)

Esta capa define el modelo matemático del grafo $G = (V, E)$.

Clase / Interfaz	Propósito y Funcionamiento	Complejidad Temporal	Complejidad Espacial
Parada	Representa un Vértice (V) . Almacena identidad y	$O(1)$ para acceso a atributos.	$O(1)$ por instancia de parada.

	coordenadas. Utiliza equals y hashCode basados en el ID para asegurar unicidad en colecciones.		
Ruta	Representa una Arista (E) dirigida. Lo más destacable es el uso de un EnumMap para almacenar múltiples pesos (tiempo, costo, etc.).	O(1) para obtener/establecer pesos.	O(1) (tamaño fijo del Enum Pond).
GrafoTransporte	Implementa el grafo mediante Listas de Adyacencia (Map<Parada, List<Ruta>>). Es la estructura más eficiente para redes de transporte.	Ver tabla de operaciones abajo.	O(V + E) para almacenar la red completa.
Pond (Enum)	Define los criterios de optimización: COSTO, TIEMPO, DISTANCIA, TRANSBORDOS.	O(1)	O(1)

Operaciones en GrafoTransporte

- **agregarParada(p)**: O(1) promedio usando HashMap.putIfAbsent.
- **agregarRuta(origen, ruta)**: O(1) al añadir a la ArrayList de adyacencia tras validar la existencia de los nodos.
- **eliminarRuta(origen, destino)**: O(E_{salida}), donde E_{salida} es el grado de salida del nodo origen, ya que debe buscar en su lista de rutas.
- **obtenerVecinos(p)**: O(1) al recuperar la lista del mapa.

2.1. Entidades Principales

Parada (Nodo): Almacena el nombre y coordenadas (x, y) para la representación visual en el Canvas de JavaFX.

Ruta (Arista): Contiene los pesos: tiempo, distancia y costo.

2.2. Justificación del Grafo

Utilizamos un Map<Parada, List<Ruta>>.

- **Por qué:** El HashMap permite un acceso promedio de $O(1)$ para recuperar los vecinos de cualquier parada, lo cual es crítico para los algoritmos de exploración como Dijkstra.

3. Análisis Asintótico de Operaciones

A continuación, se detalla la complejidad de las operaciones de gestión del grafo, donde V es el número de paradas (vértices) y E el número de rutas (aristas):

Operación	Complejidad	Justificación
Inserción de Parada	$O(1)$	Añadir una nueva clave al HashMap.
Inserción de Ruta	$O(1)$	Añadir un elemento al final de la LinkedList de adyacencia.
Eliminación de Ruta	$O(E)$	En el peor caso, se debe recorrer la lista de adyacencia del nodo origen.
Consulta de Vecinos	$O(1)$	Acceso directo mediante la clave del nodo en el mapa.
Verificación de Arista	$O(V)$	En grafos muy densos, buscar un destino específico en la lista de adyacencia.

Capa de Algoritmos (LogicPack.Algorithms)

Utiliza la interfaz Algozed<V, E> para permitir el intercambio dinámico de algoritmos de búsqueda.

DijkstraAlgo

Se utiliza para encontrar la ruta más corta cuando los pesos son no negativos.

- **Funcionamiento:** Utiliza una PriorityQueue para extraer siempre el nodo con la menor distancia acumulada.
- **Complejidad Temporal:** $O((V + E) \log V)$. La inserción y extracción de la cola de prioridad toma $\log V$.
- **Complejidad Espacial:** $O(V + E)$ para mantener los mapas de distancias, padres y la cola.

Bellman-FordAlgo

Capaz de manejar pesos negativos (como descuentos en pasajes) y detectar ciclos negativos.

- **Funcionamiento:** Relaja todas las aristas del grafo $V - 1$ veces.

- Complejidad Temporal: $O(V \cdot E)$. Es más costoso que Dijkstra pero más robusto ante pesos negativos.
- Complejidad Espacial: $O(V)$ para almacenar las distancias y predecesores.

5. Interfaz Gráfica (JavaFX)

La UI se divide en dos módulos principales para maximizar la usabilidad (30% de la nota):

1. **Panel de Control:** Implementado en FXML con validaciones para la entrada de datos (evitando pesos negativos en campos donde no corresponden).
2. **Visualizador de Grafo:** Un Pane dinámico donde:
 - Los **Nodos** se dibujan como objetos Circle.
 - Las **Aristas** son Line dirigidas.
 - Se implementó un EventHandler para que, al hacer clic en una parada, esta se seleccione como origen del cálculo.

6. Persistencia de Datos

Se implementó un sistema de carga/guardado en formato **JSON** utilizando la librería Jackson. Esto permite que el sistema mantenga la configuración de la red de transporte entre sesiones, cumpliendo con el requisito de gestión de datos.